Basi di Dati

I seguenti appunti sono a cura di ***Catalin Ceban*** *e si basano sul corso di "Basi di Dati" erogato nell'A.A. 2026 dal professore Giuseppe Perelli per il C.d.L Informatica (33503)*

13 ottobre 2026

Indice

1 La memorizzazione dei dati nel passato **3**

1.1 Sistema informativo e sistema informatico 3

1.2 La gestione dei dati prima dei DBMS 4

2 Basi di dati e DBMS **5**

2.1 Architettura ANSI/SPARC 8

2.1.1 Schema esterno 8

2.1.2 Schema logico 8

2.1.3 Schema fisico 9

2.2 Le transazioni 9

3 Progettazione di basi di dati **13**

3.1 Modelli informatici 14

3.2 Entità, collezioni e attributi 15

3.3 Le associazioni 16

3.4 Le gerarchie di classi 18

3.4.1 Vincoli su insiemi di sottoclassi 20

3.5 Le fasi per la costruzione di una base di dati 21

4 Progettazione concettuale **22**

4.1 Il modello E-R 22

4.1.1 Entità 23

4.1.2 Associazioni 23

4.1.3 Attributi 25

4.1.4 Cardinalità di associazioni 26

4.1.5 Identificatore di un'entità 28

4.1.6 Generalizzazioni 29

5 Progettazione logica **31**

5.1 Il modello relazionale 31
5.1.1 Chiavi di una relazione 33
5.2 Ristrutturazione del modello concettuale 34
5.2.1 Analisi delle ridondanze 34
5.2.2 Eliminazione delle generalizzazioni 35
5.2.3 Partizionamento/accorpamento di entità e associazioni 36
5.2.4 Scelta degli identificatori primari 38
5.3 La traduzione da schema concettuale a logico 38
5.3.1 Le entità 38
5.3.2 Le associazioni 39
5.3.3 I valori NULL 45
5.4 Vincoli di integrità 46

6 Il linguaggio SQL **49**
6.1 Domini di dati 50
6.2 Definizione delle tabelle 51

1

La memorizzazione dei dati nel passato

Si ritiene opportuno, come introduzione al corso, fornire vari esempi e una definizione abbastanza comune di "base di dati". Interagiamo con basi di dati costantemente durante la nostra quotidianità: accedendo all'applicazione della nostra banca, utilizzando i social network, effettuando un acquisto su piattaforme di E-Commerce, ...
Questi sono solo pochi degli innumerevoli esempi di basi di dati con le quali ci troviamo a contatto ogni giorno. Possiamo dunque concludere che una definizione molto poco formale e spiccola di "base di dati" sia letteralmente *un grande e organizzato contenitore di dati di alcun tipo.*

1.1 Sistema informativo e sistema informatico

Nelle situazioni sopra elencate, ma anche nella vita di tutti i giorni, è strettamente necessario tenere traccia delle informazioni riguardanti persone o cose. Pensiamo ad esempio ad un'azienda: all'interno di questa sarà necessario tenere traccia di tutti i dettagli anagrafici dei dipendenti, ma anche dei contratti, degli stipendi, ecc...
Tutti questi dati venivano e vengono ancora oggi gestiti dal cosiddetto **sistema informativo**.

Definizione 1.1.1: Sistema informativo

Il *sistema informativo* è l'insieme di quelle risorse, umane o materiali, che un'organizzazione impiega per la gestione e la memorizzazione delle informazioni.
Un banale esempio di sistema informativo potrebbe essere un cassetto contenente delle cartelle, ognuna relativa ad ogni dipendente (risorsa materiale) e la segretaria che è adibita all'accesso e al controllo di questi dati.

Come ben sappiamo però, l'epoca degli archivi fisici è da un po' passata: con l'avvento della tecnologia, e in particolare con la sua massiccia distribuzione, le organizzazioni si sono sempre di più affidate a strumenti di memorizzazione automatizzati e a sistemi per la gestione semi-automatica dei dati (gli elaboratori, per l'appunto). Nasce così un altro importante concetto, quello del **sistema informatico**.

Definizione 1.1.2: Sistema informatico

Il *sistema informatico* è l'insieme delle tecnologie utilizzate dall'organizzazione per la memorizzazione dei dati. Parliamo anche di **sistema informativo automatizzato** quando includiamo nella precedente definizione di sistema informativo anche le componenti elettroniche.

1.2 La gestione dei dati prima dei DBMS

Prima della nascita e della diffusione dei moderni sistemi di gestione di basi di dati (DBMS), i processi di organizzazione e memorizzazione delle informazioni non venivano ancora effettuati nel modo più efficiente possibile. Infatti, ogni organizzazione aveva il proprio **file privato** in cui memorizzare i dati. Utilizzando dunque un singolo file per ogni informazione, la memorizzazione dei dati avveniva in modo *sequenziale*. Dunque, per reperire un dato, era anche necessario scorrere ogni record del file finché questo non fosse stato trovato. Gli svantaggi di questa metodologia sono evidenti ad occhio nudo:

- **Ridondanza:** se lo stesso dato doveva essere condiviso tra più applicazioni, questo doveva essere duplicato (letteralmente un copia e incolla).
- **Inconsistenza:** a causa della ridondanza, aggiornando un'istanza di un record ci si poteva dimenticare di aggiornare anche le altre.
- **Dipendenza dall'applicazione:** ogni applicazione era fortemente legata alla struttura fisica delle cartelle e dei dati in suo possesso.

2

Basi di dati e DBMS

Avendo fornito una panoramica riguardo ai sistemi e alle strategie di memorizzazione utilizzate nel passato (si parla prima degli anni '70), è giusto concentrarsi sullo standard adottato da tali anni fino ad oggi. Iniziamo fornendo una definizione più formale di **base di dati**.

Definizione 2.0.1: Base di dati

Una *base di dati* è una raccolta di dati permanenti, gestita da un elaboratore elettronico.
Più nel pratico, una base di dati rappresenta un insieme di file mutualmente connessi, gestiti efficientemente in modo da rendere l'accesso ai dati il più rapido possibile.
Le basi di dati permettono la memorizzazione di due tipi di dati:

- **Metadati:** delle informazioni speciali che descrivono lo schema della base di dati (la sua struttura) e tutti i vincoli e le relazioni che intercorrono tra i vari file.
- **Dati:** le effettive informazioni che, contestualizzate, vengono inserite e memorizzate nei file. Nota importante sulla parola *contestualizzate*: un numero o una stringa da soli non significano niente. Solo una volta che identifichiamo il numero come ad esempio la matricola di uno studente e la stringa come il suo nome questi dati assumono di significato e possono essere memorizzati con un criterio all'interno dei file.

Assieme al concetto di base di dati presentiamo un altro concetto fondamentale che viaggia di pari passo con il precedente:

Definizione 2.0.2: DataBase Management System (DBMS)

Il *DBMS* è uno strumento software che si occupa principalmente di gestire grandi quantità di dati. Un buon DBMS deve inoltre fornire linguaggi e strumenti per permettere le seguenti operazioni:

- Definizione delle relazioni tra i dati.
- Inserimento/modifica/cancellazione dei dati nei file.
- Creazione dei vincoli di integrità che devono essere stabiliti tra i record in relazione.
- Memorizzazione fisica delle informazioni tramite apposite strutture dati.

Inoltre, un DBMS deve garantire due proprietà fondamentali:

- **Indipendenza fisica:** modificando l'organizzazione della base di dati, i programmi applicativi non devono risentire di modifiche o malfunzionamenti.
- **Indipendenza logica:** i programmi applicativi non devono essere modificati a seguito di una modifica nello schema logico.

Tale principio di indipendenza è fondamentale per eliminare quella dipendenza stretta tra supporto fisico e software che tanto aveva causato problemi nell'epoca pre-DBMS.

Nel mondo dell'informatica, vigono due modalità di uso principali dei DBMS:

On-Line Transaction Processing (OLTP): si tratta della metodologia classica di utilizzo dei DBMS, che prevede lo sfruttamento completo dei vincoli di integrità messi a disposizione da questo. Le organizzazioni che fanno uso di questo paradigma si concentrano maggiormente su *quali dati e come sono memorizzati*, e le operazioni svolte (*transazioni*) sono spesso molto semplici e lavorano su relativamente pochi dati.

On-Line Analytical Processing (OLAP): secondo questo paradigma è importante che i dati siano molti e vengano memorizzati in apposite strutture. Infatti, le organizzazioni che fanno uso di questa modalità tendono a sfruttare le *data warehouses* (letteralmente dei "magazzini per i dati") in quanto sono maggiormente utilizzati nell'ambito della ricerca e dell'analisi scientifica, dove non importa *come* sono organizzati i dati bensì *quali* dati sono memorizzati.

Tuttavia la vera differenza nell'uso dei DBMS venne fatta dal matematica Edgar F. Codd intorno agli anni '70, quando propose di creare un modello di DBMS che si ispirasse al concetto matematico di **relazione** (che ricordiamo essere un sottoinsieme del prodotto cartesiano tra due insiemi). Nasce così il **modello relazionale**.

Definizione 2.0.3: DBMS relazionale

Un DBMS che implementa il modello relazionale organizza i propri dati all'interno di n-uple ordinate, dette **record**, in cui ogni elemento del record rappresenta una particolare informazione.

Nella definizione di una base di dati relazionale bisogna anche definire uno **schema** secondo il quale organizzare questi dati. Tale schema non è altro che la *struttura logica* della tabella, cioè una n-upla di quelli che sono gli attributi (o più informalmente, le colonne) della tabella. Infine, possiamo dire che una tabella contiene tutti record di tipo omogeneo, cioè tutti record che hanno la stessa struttura (o, appunto, lo stesso schema).

All'interno del modello relazionale, due record non vengono collegati tramite puntatori come ad esempio nel modello gerarchico o reticolare (vedi dopo) bensì viene stabilito un collegamento tra due attributi dei due record che devono necessariamente essere dello stesso tipo e contenere lo stesso valore. Intuitivamente si può pensare che tale procedimento causi una forte ridondanza dei dati, ma tale osservazione è parzialmente errata in quanto:

- Un buon DBMS deve mettere a disposizione gli strumenti per la creazione di tali collegamenti e controllare che questi non vengano violati (**vincoli di integrità**).
- Ogni record è memorizzato **una e una sola volta** sul disco fisico. Ciò vuol dire che se un record studente è associato ad una classe, non verrà duplicato alcun record per stabilire il collegamento, ma semplicemente verranno collegati i due record già esistenti. Tale ridondanza deve però essere gestita, come detto anticipatamente, dal DBMS.

Matricola	Cognome	E-Mail
2671072	Rossi	rossi@sapienza.it

2623072	Bianchi	bianchi@sapienza.it
2623272	Verdi	verdi@sapienza.it

Sopra è presentato un esempio di come vengono organizzati i dati all'interno di una tabella nel modello relazionale. Come possiamo vedere ogni record ha la stessa struttura: matricola, cognome, e-mail. Questa struttura inoltre è proprio lo *schema della tabella* (inclusi vari vincoli di integrità stabiliti).

Tuttavia il modello relazionale non fu e ancora oggi non è l'unico modello per la rappresentazione dei dati.

Modello reticolare: i dati vengono presentati come una collezione di record di tipo omogeneo (simile al relazionale). Tuttavia, le effettive *relazioni* tra i vari record venivano fatte tramite dei *puntatori* da un record ad un altro.

Modello gerarchico: si tratta di una sottoclasse del modello reticolare, e prevede l'organizzazione dei dati in una struttura simile a quella di un albero informatico, dove ogni nodo (record) ha solo un genitore.

2.1 Architettura ANSI/SPARC

Nello studio dei sistemi di gestione di basi di dati, risulta particolarmente utile ed efficace distinguere tre differenti livelli di descrizione dei dati, noti comunemente come schemi. Tale suddivisione permette di garantire l'indipendenza dei dati e di gestire la complessità strutturale del sistema. Vediamo più nel dettaglio ciascun livello componente questa architettura.

2.1.1 Schema esterno

Si tratta della porzione più superficiale dell'architettura, quella con cui saranno a contatto gli utenti che non dovranno avere nulla a che fare con la gestione dei dati, ma solo con il loro utilizzo (ad esempio, la segretaria di un ufficio). Tale schema predispone anche che ogni utente sia in grado di visualizzare solo *una porzione* delle informazioni contenute nella base di dati, in base ai suoi permessi.

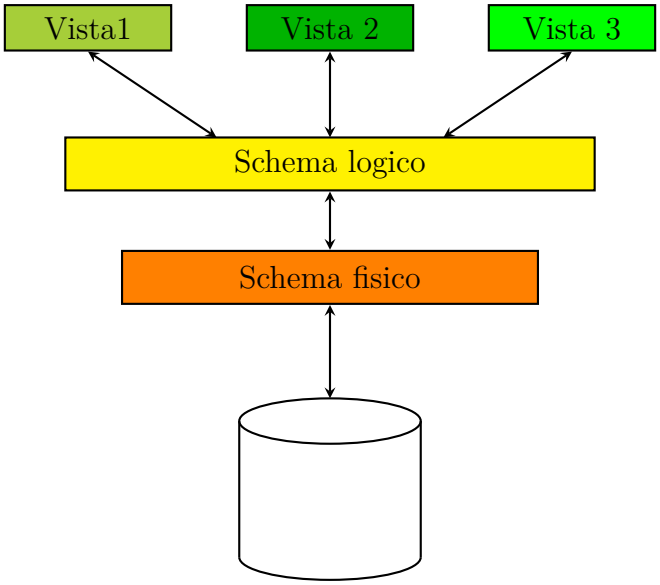
2.1.2 Schema logico

Questo livello descrive la struttura degli insiemi di dati e delle relazioni tra loro, secondo il modello adottato dal DBMS. Nota che questo livello si occupa di definire lo schema

dei dati, ma non ha alcuna influenza su come questi vengono memorizzati fisicamente in memoria. Un esempio di schema logico può essere il seguente, che definisce la struttura di una tabella all'interno della base di dati di un'università; `Studenti(matricola char(8), cognome char(20), annoDiNascita int)`

2.1.3 Schema fisico

L'ultimo e più profondo livello si occupa di gestire l'effettiva memorizzazione dei dati all'interno dei supporti di memorizzazione, fornendo le strutture dati necessarie per facilitarne l'accesso (in particolare i B-Alberi). Ad esempio, l'amministratore della base di dati potrebbe decidere che i dati vadano *memorizzati* in maniera sequenziale (i record vengono messi uno dopo l'altro) e l'accesso a questi venga fatto tramite un indice sull'identificativo univoco del record.



2.2 Le transazioni

Un buon DBMS deve inoltre gestire anche una serie di operazioni sensibili di inserimento, modifica e cancellazione di dati, in modo che queste vengano effettuate senza contrattempi o errori.

Definizione 2.2.1: Transazione

Una **transazione** è una sequenza di operazioni di lettura e scrittura in memoria che deve rispettare le seguenti proprietà **ACID**:

- **Atomicity:** deve essere una sequenza **indivisibile** di operazioni. In caso di verifica di un qualsiasi errore nel mezzo di una transazione porta questa ad

essere marcata come *non valida*, in quanto le operazioni devono essere eseguite **tutte o nessuna**.

- **Consistency:** bisogna lasciare i dati modificati in uno stato corretto che non viola i vincoli imposti precedentemente dal DBMS.
- **Isolation:** ogni transazione deve essere **indipendente**. Dunque durante esecuzione concorrente di più transazioni, gli effetti di una non devono dipendere dalle altre.
- **Durability:** gli effetti di una transazione devono essere **permanenti** per quanto riguarda i dati modificati (ovvero le modifiche devono necessariamente essere applicate anche su disco per mantenere la sincronizzazione dei dati).

Vediamo di seguito un esempio di due transazioni che vengono eseguite bene o male nello stesso momento:

- **Transazione 1**: "Accredita 1000 € sul conto corrente $c1$"
- **Transazione 2**: "Accredita 500 € sul conto corrente $c1$"

Supponiamo che il valore iniziale del saldo del conto $c1$ sia pari a 2000 €. Intuitivamente, al termine di entrambe le transazioni, ci aspetterions un valore finale di 3500 €. Vediamo invece cosa può accadere se le operazioni vengono interlacciate nel tempo senza opportune sue protezioni.

Osservazione 2.2.1

Una volta letto un valore, ogni transazione lo modifica nel **proprio** spazio di memoria di lavoro; altrimenti si rischiano modifiche irreversibili ai dati nello schema.

Analizziamo la seguente tabella di esecuzione temporale delle due transazioni:

Transazione 1	Tempo	Transazione 2
cerca $c1$	$t1$	
	$t2$	cerca $c1$ ← *qui non prende il valore aggiornato* +1000

modifica *saldo* in *saldo*+1000	$t3$	
	$t4$	modifica *saldo* in *saldo*+500

Come si può vedere dalla tabella, l'esecuzione concorrente di due transazioni, in questo caso, risulta in un esito errato, in quanto ogni transazione effettua modifiche sulla **sua** porzione di dati, e non su quella effettivamente memorizzata, il che causa problemi di sincronizzazione.

Definizione 2.2.2: Serializzabilità

Nel caso di esecuzioni concorrenti di più transazioni, l'effetto complessivo deve essere equivalente a quello di un'esecuzione seriale, ovvero con le transazioni eseguite *una dopo l'altra*, senza alcun interfogliamento delle loro operazioni elementari.

Chiaramente la serializzabilità non è sempre semplice da implementare, e si fa infatti uso di protocolli progettati per garantire a priori la correttezza di qualunque schedule di transazioni che li adottano. Queste tecniche e protocolli non saranno tuttavia argomento di questo corso, per cui si lascia al lettore la voglia di approfondire.

Continuando l'analisi delle funzionalità offerte da un DBMS, è fondamentale esaminare gli strumenti messi a disposizione per la gestione e lo sviluppo. Tali funzionalità non si rivolgono unicamente agli utenti finali che eseguono interrogazioni sui dati, ma coprono ruoli cruciali come l'amministratore della base di dati e lo sviluppatore di applicazioni.

Concludendo questo capitolo, se volessimo elencare i vantaggi e gli svantaggi nell'utilizzo di un DBMS *ben progettato* per la memorizzazione dei dati, otterremmo un simile resoconto.

Osservazione 2.2.2: Vantaggi nell'uso di un DBMS

1. Indipendenza fisica e logica tra dati e applicazioni che li usano.
2. Memorizzazione e recupero efficiente dei dati.
3. Vincoli di integrità sui dati.

Il principale **svantaggio** nell'utilizzo di un DBMS risiede nella stretta necessità di definire uno schema di dati prima di poterli memorizzare. Tale punto diventa uno svantaggio nel

momento in cui si vogliono memorizzare, ad esempio, piccole quantità di dati (ad esempio se vogliamo memorizzare il contenuto di un libro), oppure non ci interessa *come* questi dati vengano memorizzati.

3

Progettazione di basi di dati

Progettare una base di dati significa progettare sia la **struttura dei dati** sia le **applicazioni** che vi operano sopra. Tra queste due componenti, la **progettazione dei dati** rappresenta l'attività più importante e delicata. Tale progettazione non è un'attività fine a sé stessa, bensì rientra nel **ciclo di vita di un sistema informativo**.

Definizione 3.0.1: Ciclo di vita di un sistema informativo

Il *ciclo di vita di un sistema informativo* è quella sequenza di attività svolte da analisti, progettisti e utenti nello sviluppo dei sistemi informativi. Viene detto *ciclo* in quanto ogni fase non è una tantum, bensì può essere ripetuta più volte apportando le opportune modifiche al fine di raggiungere il risultato finale sperato.

Tale ciclo si articola nelle seguenti fasi:

1. **Studio di fattibilità:** definizione delle proprietà, stime dei costi, valutazione dei tempi richiesti dal progetto.
2. **Raccolta e analisi dei requisiti**
3. **Progettazione e implementazione:** modellazione concettuale e logica dei dati che si vogliono memorizzare e implementazione tramite un apposito linguaggio DBMS.
4. **Validazione e collaudo:** effettuazione dei test per verificare che il sistema risponda correttamente alle esigenze.

La modellazione di una base di dati si colloca a *metà* tra la raccolta e l'analisi dei requisiti e la progettazione effettiva.

3.1 Modelli informatici

Prima di modellare la base di dati, bisogna decidere a quale modello informatico/matematico rifarsi per la memorizzazione di questi. Un modello deve presentare le seguenti caratteristiche:

- Deve essere **la rappresentazione di un certo fatto concreto**.
- Tale rappresentazione deve essere descritta mediante un **linguaggio formale** (dunque che segue delle regole ben definite).
- Individuazione delle strutture da utilizzare per organizzare i dati di interesse e le relazioni tra questi.
  Ad esempio, il modello relazionale di cui abbiamo discusso prima utilizza il concetto matematico di *relazione* per organizzare i dati.

Possiamo definire due tipologie di modelli fondamentali nella progettazione di una base di dati:

Definizione 3.1.1: Modello logico e modello concettuale

Il **modello logico** di una base di dati rappresenta la struttura dei dati e come questi vengono memorizzati. Si potrebbe dire che corrisponde allo schema logico visto precedentemente nel modello ANSI/SPARC, in quanto si colloca indipendente dalla memorizzazione fisica dei dati, ma è strettamente legato dal tipo di modello informatico che abbiamo scelto di adottare per la memorizzazione di questi.

Il **modello concettuale** è tutto il contrario: deve essere completamente indipendente dalle modalità di realizzazione, e ha lo scopo di rappresentare la realtà descritta dal problema, tramite un preciso linguaggio formale (ad esempio i diagrammi UML, ma anche il modello **Entity-Relationship (E-R)**)

Non esiste un modello corretto: la stessa realtà può essere modellata in modi diversi a seconda dell'interpretazione che si dà al problema. Tuttavia deve essere seguito uno specifico "percorso mentale" nella progettazione di una base di dati:

1. **Aspetto ontologico:** quanto conosco del dominio del discorso? Quanto sono familiare con l'ambito di discussione?
   Durante questa fase, detta anche fase di *analisi del problema*, devo delineare quali sono le entità la cui realtà deve essere modellata e come si relazionano tra loro.

2. **Aspetto logico:** con quale concetto astratto voglio realizzare le entità del modello? (es. relazionale, gerarchico, reticolare, ...)
   Durante questa fase definisco le effettive relazioni tra le entità.

3. **Aspetto linguistico:** con quale linguaggio formale definiamo il modello? (es. SQL/algebra relazionale)

4. **Aspetto pragmatico:** come memorizzo i dati nei supporti di memorizzazione fisica?

3.2 Entità, collezioni e attributi

Nella descrizione delle fasi da seguire durante la progettazione di una base di dati abbiamo citato un termine importante e fondamentale nella modellazione concettuale della realtà di riferimento: **entità**.

Definizione 3.2.1: Entità

Le *entità* di un problema sono quegli oggetti a cui il problema dà particolare rilevanza, e dei quali ci interessa rappresentare alcuni fatti a loro associati, detti per l'appunto **attributi (o proprietà) dell'entità**.
Ogni entità appartiene ad un **tipo** che ne specifica la natura. Ad esempio l'entità `Studente` appartiene alla categoria `Persona` (possiamo vederla come l'ereditarietà nella programmazione ad oggetti).
Una **collezione** di entità è invece un insieme variabile nel tempo di entità tutte dello stesso tipo. Ad esempio, se pensiamo a tutti gli studenti di un'università, quella rappresenta una collezione di entità di tipo `Studente`.

Supponiamo di dover creare la base di dati di un sistema di memorizzazione degli studenti, professori ed esami. Possiamo individuare nello studente un'**entità**, in quanto si tratta di uno degli oggetti fondamentali descritti dal problema, e nella sua matricola, cognome e data di nascita gli **attributi** fondamentali che lo descrivono.

Un entità o un attributo non mantengono il loro status costantemente all'interno di ogni analisi e di ogni problema: in una realtà potrebbe essere messa più importanza su un oggetto, spingendoci dunque a farlo diventare entità, mentre in un'altra tale importanza potrebbe mancare, rilegando così l'oggetto ad un semplice attributo. Ad esempio, proviamo a rappresentare una biblioteca: se ci interessasse modellare solo i libri, senza curarci degli autori, potremmo avere uno schema logico di questo tipo `Libro(titolo, annoDiPubblicazione, casaEditrice, autore)`. Se invece fossimo interessati anche agli

autori, renderemmo quell'oggetto una vera e propria entità con nome e cognome, anziché lasciarlo un semplice attributo.

3.3 Le associazioni

Definizione 3.3.1: Associazione

Un'**istanza di associazione** è un fatto che collega due o più entità, stabilendo un legame logico fra loro.
Matematicamente, definiamo $R(X, Y)$ un'associazione fra due collezioni di entità X ed Y un insieme di istanze di associazione tra elementi $x \in X$ e $y \in Y$. Il prodotto cartesiamo $X \times Y$ è detto **dominio dell'associazione**.

Generalmente, nella modellazione concettuale, le associazioni sono descritte da un *verbo* che lega le varie entità coinvolte. Chiaramente tale verbo deve essere esplicativo della natura dell'associazione, e per tale ragione si devono evitare verbi come "essere" o "avere" oppure verbi modali.
Un esempio di associazione possiamo vederlo anche con il precedente esempio della biblioteca: le entità `Libro` e `Autore` sono collegate tra loro dall'associazione `SCRIVERE`, in quanto è l'autore a scrivere il libro, e il libro viene scritto dall'autore.

Un'associazione è caratterizzata dalle seguenti proprietà strutturali:

- **Molteplicità:** prese le due collezioni di entità X ed Y e un'associazione $R(X, Y)$, indica quante istanze di un elemento $x \in X$ sono associate ad uno e un solo elemento $y \in Y$.
  Formalmente, tale proprietà è descritta dal **vincolo di univocità**:
  - Un'associazione si dice **univoca** rispetto ad X se per ogni elemento $x \in X$ esiste **al più** un elemento $y \in Y$ che è associato ad x.
  - Se tale vincolo fallisce (ovvero se esiste **almeno** un elemento $x \in X$ a cui è associato più di un elemento $y \in Y$) l'associazione si dice **multivalore**.

  Facendo un esempio, un'associazione del tipo $R(X, Y) = (1 : N)$ è multivalore su X e univoca su Y. Viceversa, se avessimo scritto $R(X, Y) = (N : 1)$, sarebbe stata univoca su X e multivalore su Y. Possiamo leggerla "a croce", diciamo: la molteplicità dell'elemento più a sinistra è il valore più a destra, mentre quella dell'elemento più a destra è il valore più a sinistra. Come nelle proporzioni, gli esterni vanno con gli esterni e gli interni con gli interni.

- **Totalità:** indica se l'associazione tra un elemento $x \in X$ e un elemento $y \in Y$ è opzionale oppure no. Formalmente:
  - Un'associazione $R(X, Y)$ si dice **totale** se, per ogni elemento $x \in X$ esiste *almeno* un elemento $y \in Y$ associato ad x (questa proprietà è conosciuta meglio come **obbligatorietà del valore**).
  - Tale associazione si dice invece **parziale** quando il vincolo imposto precedentemente fallisce, ovvero quando ad un valore $x \in X$ *non è detto* che vi sia associato un valore $y \in Y$ (questa proprietà è più conosciuta come **opzionalità**).

  Ad esempio, pensiamo ad un'associazione del tipo `Possedere(Persona, AutomobileConTarga)`. Riprendendo il discorso sulla molteplicità, questa sarà del tipo `(1:N)`, in quanto una persona può possedere più di un'automobile con targa, però una singola automobile con targa è posseduta da una sola persona. Vedendo invece il vincolo di totalità, quest'associazione risulta essere *opzionale* sulla Persona, in quanto non è detto che una persona possegga un'automobile, mentre è *totale* su automobile, in quanto un'automobile già targata è sicuramente proprietà di qualcuno.

Vediamo adesso una delle numerose rappresentazioni grafiche con cui possiamo rappresentare le associazioni, e delle particolarità che le contraddistinguono:

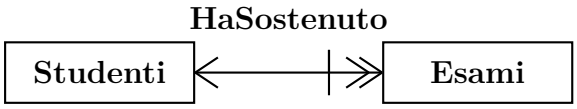


Figura 3.1: Relazione **HaSostenuto**

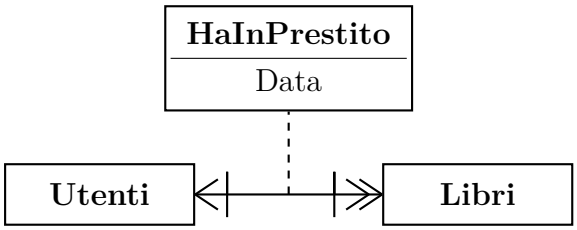


Figura 3.2: Relazione con attributi. L'attributo *Data* è agganciato alla relazione stessa.

Descriviamo adesso tutte le informazioni ricavabili dalla notazione presa in esempio:

- Il singolo `>` oppure `<` significa che la relazione è **univoca** verso quel lato. Vedendo la figura 3.1 vediamo come in questa specifica modellazione un esame venga sostenuto da uno e un singolo studente.

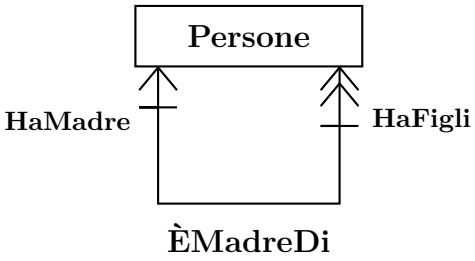


Figura 3.3: Relazione ricorsiva. L'entità **Persone** si relaziona con sé stessa tramite i ruoli *HaMadre* e *HaFigli*.

- Il doppio » oppure « significa che da quel lato la relazione è **multivalore**. Sempre prendendo in esempio gli studenti, uno studente può aver sostenuto più volte lo stesso esame. Anche con `Utenti - Libri`, vediamo come un utente prende in presito più libri, mentre un libro viene preso in prestito solo da un utente alla volta.
- La barra verticale `|` vicino alla molteplicità indica che da quel lato la relazione è **parziale**, dunque opzionale. Ad esempio vediamo come, nel caso `Studenti - Esami` il singolo studente potrebbe ancora non aver mai sostenuto quell'esame, mentre l'esame stesso, essendo registrato, è sicuramente stato sostenuto da qualche studente.

3.4 Le gerarchie di classi

Spesso le classi di entità sono organizzate in una gerarchia di *specializzazione/generalizzazione*. Una classe della gerarchia minore di altre viene detta **sottoclasse**, mentre le altre prendono il nome di **superclassi**. Il concetto è lo stesso alla base dell'ereditarietà nella programmazione ad oggetti. Per fare un esempio pertinente, la classe degli *studenti universitari* può è una generalizzazione (o sopraclasse) delle seguenti classi:

- Matricole e laureandi.
- Studenti in corso e studenti fuori corso.
- Studenti locali e studenti fuori sede.
- Studenti maschi e studentesse.

Fra i tipi oggetto è definita una **relazione di sottotipo** che regola la struttura gerarchica dei dati. Per il lettore questa è semplicemente una formalizzazione matematica del concetto di *polimorfismo* all'interno della programmazione ad oggetti.

Definizione 3.4.1: Relazione di sottotipo

Fra i tipi oggetto viene stabilita una relazione di sottotipo che soddisfa le seguenti proprietà fondamentali:

- È **asimmetrica**, **riflessiva** e **transitiva** (definendo così una relazione di ordine parziale).
- **Principio di sostitutività**: Se T è sottotipo di T', allora gli elementi appartenenti al tipo T possono essere usati in qualsiasi contesto in cui possano apparire valori di tipo T'.

  In particolare, ciò implica che:

  – Gli elementi di T possiedono tutte le proprietà degli elementi di T'.

  – Per ogni proprietà p in T', il suo tipo associato in T deve essere un sottotipo del suo tipo in T'.

Facendo nuovamente riferimento alla programmazione ad oggetti, anche nella modellazione concettuale possiamo definire degli oggetti (le entità del problema che vogliamo modellare) a partire da altri già esistenti. Questi nuovi oggetti erediteranno dal proprio genitore (o *dai propri genitori*, in quanto viene supportata l'ereditarietà multipla) tutti i suoi attributi, e potranno anche definirne di nuovi o ridefinire gli esistenti.

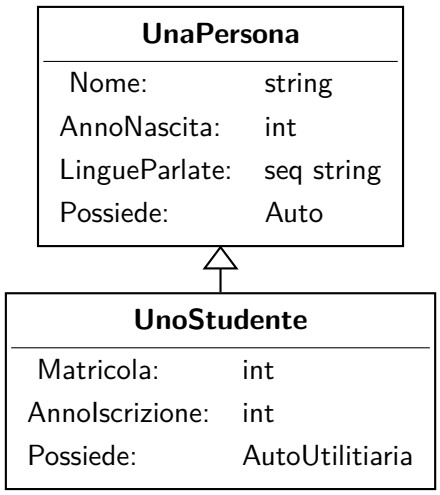


Figura 3.4: Ereditarietà tra classi. La classe **UnoStudente** eredita da **UnaPersona**, specializzando il tipo dell'attributo *Possiede*.

Fra le classi può essere definita una relazione di **sottoclasse**, detta anche di **sottoinsieme**, caratterizzata dalle seguenti proprietà:

Definizione 3.4.2: Relazione di sottoclasse

La **relazione di sottoclasse** è definita dalle seguenti proprietà:

- È **antisimmetrica**, **riflessiva** e **transitiva**.
- Se C è sottoclasse di C', allora il tipo degli elementi di C è sottotipo del tipo

degli elementi di C' (**vincolo intensionale**).

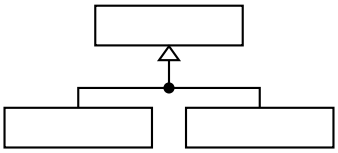
- Se C è sottoclasse di C', allora gli elementi di C sono un sottoinsieme degli elementi di C' (**vincolo estensionale**).

3.4.1 Vincoli su insiemi di sottoclassi

Per gli insiemi delle sottoclassi (i domini dei valori che tali classi possono assumere) vengono anche stabiliti dei vincoli.

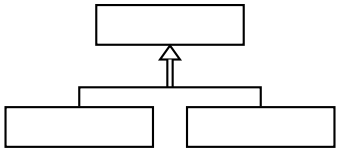
Definizione 3.4.3: Vincolo di disgiunzione

Gli elementi di una sottoclasse non possono far parte di un'altra sottoclasse.
Sostanzialmente stiamo dicendo che, avendo delle classi che sono specializzazioni di una sopraclasse, tali sottoclassi **non avranno nulla in comune**, e dunque un elemento di una sottoclasse non avrà nulla a che vedere con un elemento di un'altra sottoclasse.
Pensiamo ad esempio di avere una classe `Persona` che ha come sottoclassi `Maggiorenni` e `Minorenni`. Tra queste sottoclassi vige il vincolo di disgiunzione, in quanto una persona non potrà **mai** essere contemporaneamente sia minorenne che maggiorenne. D'altro canto, se avessimo una classe `Persona` con sottoclassi `Programmatore` e `Maggiorenne`, tra queste due sottoclassi non vige il vincolo di disgiunzione in quanto non possiamo categoricamente dire che nessun programmatore è maggiorenne o viceversa. Formalizzando in maniera matematica questo vincolo, possiamo dire che *i domini delle due sottoclassi sono due insiemi la cui intersezione è nulla.*



Definizione 3.4.4: Vincolo di copertura

L'unione degli elementi delle sottoclassi contiene **tutti** gli elementi della sopraclasse.
Mantenendoci sempre sull'esempio fatto in precedenza con `Persona, Maggiorenne, Minorenne`, tra le due sottoclassi è presente un vincolo di copertura, in quanto una persona **o** è maggiorenne **o** è minorenne. Necessariamente una delle due. In modo più matematico e formale, possiamo dire che *l'unione dei domini delle sottoclassi è uguale al dominio della sopraclasse.*



Se tra le sottoclassi di una gerarchia si verificano contemporaneamente il vincolo di disgiunzione e quello di copertura, si dice che l'insieme delle sottoclassi è una **partizione** della classe padre.

3.5 Le fasi per la costruzione di una base di dati

Il ciclo di vita e il processo di sviluppo di una base di dati si articola nelle seguenti fasi principali:

- **Analisi dei requisiti**: consiste nell'analizzare il problema e comprenderne a fondo le specifiche fornite dall'utente o committente, tipicamente in linguaggio naturale.
- **Progettazione**: si suddivide a sua volta nella progettazione dei dati e nella progettazione delle applicazioni.
  - **Progettazione concettuale**: ha lo scopo di individuare le entità di interesse e le associazioni tra di esse, formalizzandole tramite un apposito modello concettuale ad alto livello (ad esempio il modello E-R o i diagrammi delle classi UML) che serve a descrivere la realtà di riferimento da un punto di vista abbastanza astratto, in maniera totalmente indipendente da quella con cui saranno organizzati i dati dal DBMS.
  - **Progettazione logica**: consiste nella traduzione dello schema concettuale nel modello logico scelto (ad esempio il modello relazionale), definendo puntualmente le strutture dati logiche e le relazioni intercorrenti tra esse.
  - **Progettazione fisica dei dati**: riguarda gli aspetti implementativi di basso livello, definendo l'organizzazione fisica e le strutture di memorizzazione dei dati su disco (file di memorizzazione, indici, metodi di accesso).
  - **Progettazione delle applicazioni**: definizione dell'architettura e della logica dei programmi software che interagiranno con la base di dati.
- **Realizzazione**: fase in cui il database viene effettivamente implementato sul DBMS prescelto, popolato e integrato con le applicazioni software.

4

Progettazione concettuale

Nel corso del seguente capitolo analizzeremo la prima fase della progettazione di una base di dati, forse la più fondamentale, che ci permetterà di tracciare le basi per la creazione delle relazioni e delle tabelle in memoria.

Durante la fase di progettazione concettuale, in cui si va a delineare il **modello concettuale** derivato dall'analisi del problema, è prevalente l'**aspetto intensionale della progettazione**.

Definizione 4.0.1: Aspetto intensionale

L'aspetto intensionale è quel lato della progettazione che definisce quali entità esistono, quali attributi hanno e quali vincoli devono rispettare. E' **invariante nel tempo**.

Il modello concettuale più diffuso è sicuramente il **modello E-R**, del quale parleremo maggiormente nella prossima sezione.

4.1 Il modello E-R

Il modello E-R è un modello grafico utilizzato per descrivere la struttura astratta di una base di dati, indicando gli oggetti coinvolti e le relazioni tra questi. Mette le basi su alcuni costrutti fondamentali.

4.1.1 Entità

Definizione 4.1.1: Entità

Classi di oggetti omogenei tra loro e con esistenza autonoma. Ciò vuol dire che un'entità, per esistere, non deve essere necessariamente collegata o dipendente da qualcos'altro. Sono, insieme alle associazioni, l'aspetto più importante del problema, dato che, informalmente parlando, le entità si trasformeranno nelle *tabelle* durante la progettazione del modello logico. Un esempio di entità potrebbe essere un `Professore` o un `Corso` nell'ottica di una base di dati universitaria. L'entità è quell'oggetto a cui, nella descrizione della realtà di riferimento, si attribuisce particolare importanza e a cui è richiesto che vengano fatte afferire delle informazioni (gli attributi).

Ogni entità ha un **nome identificativo** (che può essere qualsiasi, ma ovviamente si tende a scegliere nomi che facciano capire cosa si sta modellando) che per convenzione si scrive al **singolare**.

E' importante distinguere il concetto di entità da quello di **istanza**, in quanto quest'ultima non rappresenta una classe di oggetti dello stesso tipo bensì **il singolo oggetto, il singolo elemento della classe**. Nel modello concettuale rappresentiamo le entità, non le singole istanze.

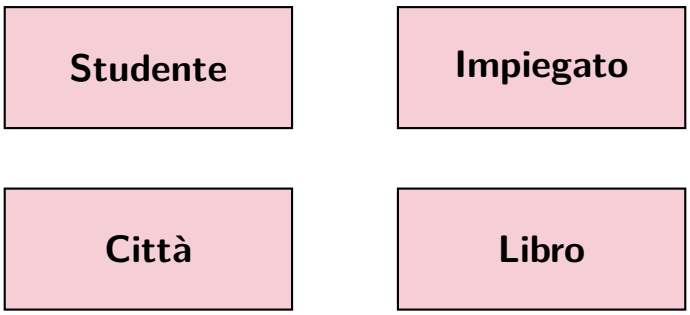


Figura 4.1: Un esempio della rappresentazione grafica delle entità nel modello E-R

4.1.2 Associazioni

Definizione 4.1.2: Associazione

Legame logico che si stabilisce tra due o più entità che hanno qualcosa in comune. Queste *non* sono autonome, in quanto affinché esista un'associazione è necessario ci siano almeno 2 entità coinvolte in questa. Mantenendo l'esempio di `Professore` e `Corso`, un'associazione che li lega potrebbe essere `Insegnamento`.

A seconda della convenzione utilizzata, le associazioni potrebbero essere indicate da un sostantivo oppure da un verbo.

Anche qui, è importante distinguere tra associazione e **occorrenza**, in quanto quest'ultima individua il singolo elemento della classe di legami, identificato dalle istanze delle entità che collega. In generale un'associazione si dice **n-aria** quando collega contemporaneamente n entità.

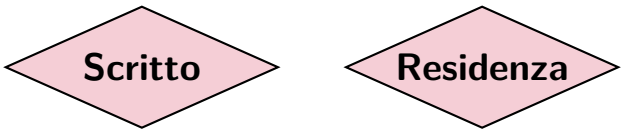


Figura 4.2: Rappresentazione grafica di associazioni senza le entità coinvolte.

Vediamo adesso alcune categorie particolari di associazioni che si potrebbero incontrare nella scrittura del modello E-R.

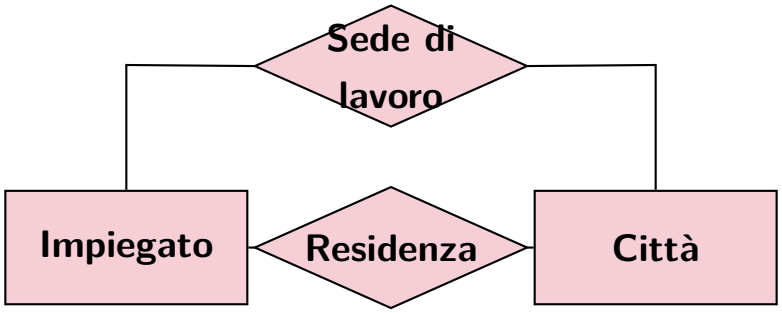


Figura 4.3: Multiple associazioni sulle stesse entità

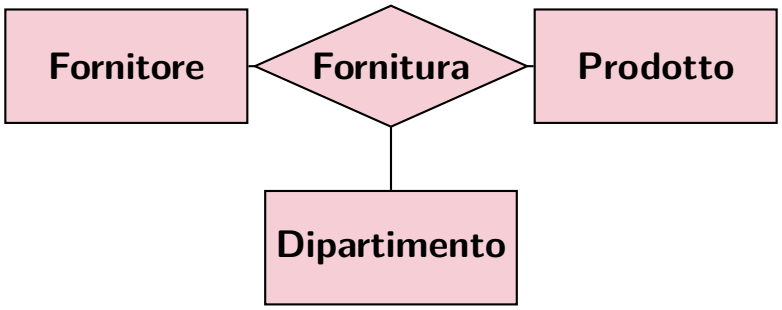


Figura 4.4: Associazione n-aria (in questo caso ternaria).

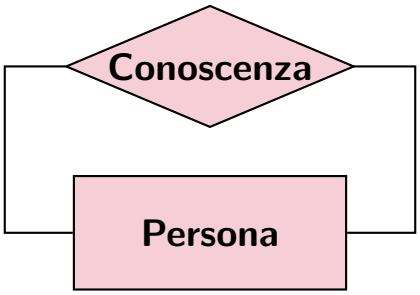


Figura 4.5: Associazione ricorsiva, cioè che coinvolge un'entità con sé stessa.

4.1.3 Attributi

Definizione 4.1.3: Attributo

Proprietà elementare, caratteristica di un'entità o di un'associazione. Si potrebbe pensare agli attributi del modello concettuale in modo similare agli attributi di classe nella programmazione ad oggetti.

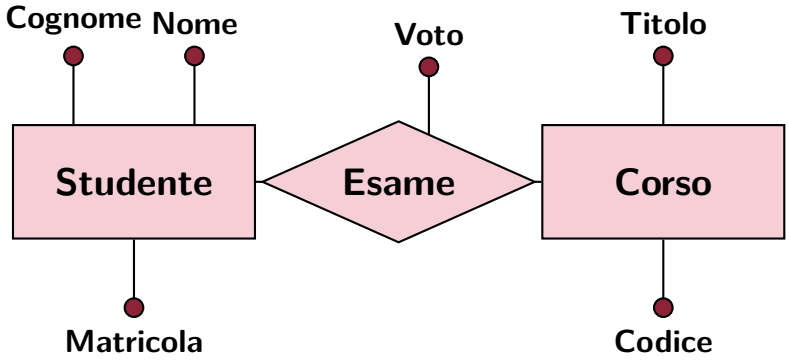


Figura 4.6: Modello E-R con entità, associazioni, e i loro attributi.

Gli attributi possono anche essere **composti**, cioè attributi che raggruppano più informazioni insieme. Un esempio di attributo composto potrebbe essere l'indirizzo di residenza di una persona, in quanto l'informazione indirizzo può essere ulteriormente scomposta in *Via*, *Numero civico* e *CAP*, ad esempio. In un'applicazione la scelta di mantenere un attributo composto anziché suddividerlo nelle sue componenti atomiche dipende dall'importanza che quell'attributo in seguito all'analisi del problema.

4.1.4 Cardinalità di associazioni

Definizione 4.1.4: Cardinalità di associazioni

Si tratta di una coppia di valori che si attribuisce ad ogni entità coinvolta in un'associazione. Si tratta di una rappresentazione grafica dei concetti di **univoco** e **multivalore** visti precedentemente.

Specificano il numero minimo e massimo di occorrenze dell'associazione a cui un'istanza della prima entità può essere collegato rispetto ad istanze della seconda entità.

Vediamo adesso più nel dettaglio le varie tipologie di cardinalità:

- **Partecipazione minima**
  - **0**: indica che la partecipazione di un'istanza dell'entità di partenza all'associazione è **opzionale**. Ciò vuol dire che potrebbe esserci anche una singola occorrenza dell'entità di partenza che non è associata a nessuna occorrenza dell'entità di arrivo. E' il corrispettivo dell'associazione **parziale** che avevamo già visto.
  - **1**: indica che ogni istanza dell'entità di partenza è associata ad almeno un'occorrenza dell'altra entità. E' il corrispettivo dell'associazione **totale** che avevamo già visto.
- **Partecipazione massima**:
  - **1**: indica che ogni occorrenza dell'entità di partenza è collegata al massimo ad un'occorrenza dell'entità di arrivo. E' il corrispettivo dell'associazione **univoca** che avevamo già visto.
  - **N**: indica che ogni occorrenza dell'entitò di partenza può essere collegata a più occorrenze dell'entità di arrivo. E' il corrispettivo dell'associazione **multivalore** che avevamo già visto.

Con riferimento alle cardinalità appena mostrate, possiamo distinguere varie tipologie di associazioni:

- **Uno ad uno**, si verifica quando la cardinalità dell'entità è del tipo (1, 1).
- **Uno a molti**, si verifica quando la cardinalità è del tipo (1, N).
- **Molti a molti**, si verifica quando entrambe le entità coinvolte hanno cardinalità (0, N) oppure (1, N).



Figura 4.7: Relazione tra un tutor e un docente, che si legge nel seguente modo: Un tutor fa il tutoraggio ad 1 massimo 2 docenti. Un docente può avere da 0 a 5 tutor che fanno tutoriaggio.

E' possibile delineare la cardinalità anche per gli attributi: quando questa non viene indicata è implicitamente (1, 1).

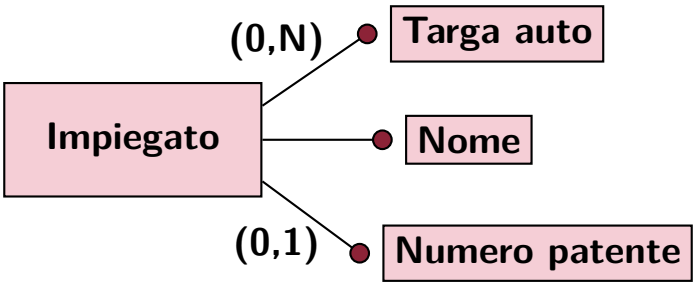


Figura 4.8: Esempio di cardinalità su attributi. Come per le associazioni, scrivere 0 come cardinalità minima indica che l'attributo non è obbligatorio.

4.1.5 Identificatore di un'entità

Definizione 4.1.5: Identificatore

L'identificatore è quell'attributo o quell'insieme di attributi che permettono di identificare univocamente un'istanza di entità.
Questi identificatori possono essere costituiti da:

- Uno o più attributi interni all'entità.
- Uno o più attributi esterni ottenuti attraverso associazioni.

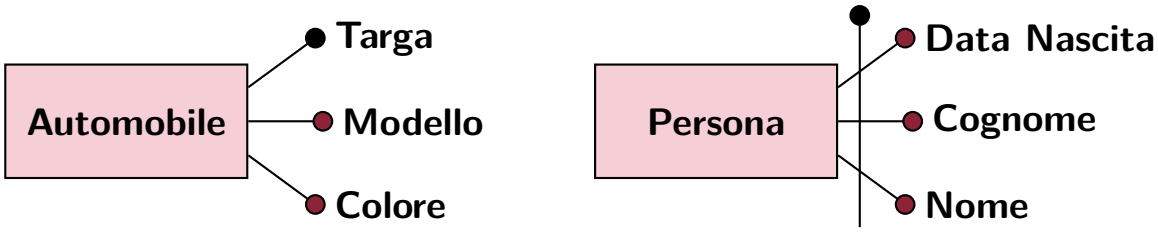


Figura 4.9: Esempio di identificatori interni semplici e composti.

Osservazione 4.1.1: Natura degli identificatori

- Un entità deve avere **almeno** un identificatore, ma può averne anche più di uno.
- Un identificatore esterno è possibile solo per entità che partecipano ad un'associazione con cardinalità (1, 1).

- Per le associazioni non occorre un identificatore in quanto non rappresentano un fatto bensì un legame logico.

4.1.6 Generalizzazioni

Definizione 4.1.6: Generalizzazione

Una generalizzazione è un costrutto che mette in relazione una o più entità $E_1, E_2, \ldots, E_n$ con un'entità E che le comprende come casi particolari. In poche parole, le entità $E_1, \ldots, E_n$ sono delle sottoclassi (o **specializzazioni**) dell'entità E, e dunque ereditano tutte le caratteristiche dell'entità padre potendo poi definirne di nuove.

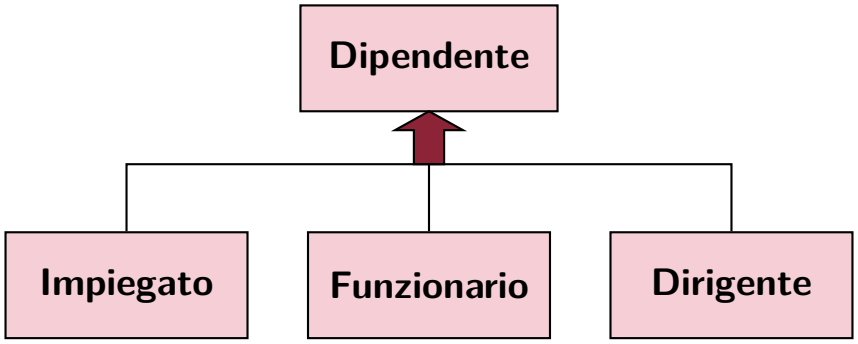


Figura 4.10: Gerarchia di generalizzazione.

Una generalizzazione può essere

Totale: quando ogni occorrenza dell'entità padre appartiene ad almeno un'occorrenza di una delle specializzazioni (in parole povere, quando un istanza del padre è del tipo di almeno un figlio). Un esempio potrebbe essere, `Persona <- Maggiorenne, Minorenne`. In questo caso è totale in quanto una persona è maggiorenne oppure minorenne.

Parziale: quando esiste almeno un'occorrenza dell'entità padre che non appartiene ad alcuna categoria. Ad esempio `Animale <- Cane, Gatto`. Ovviamente cani e gatti non sono gli unici animali esistenti.

Esclusiva: quando ogni occorrenza del padre può appartenere al massimo ad una categoria. Vale l'esempio di prima dei maggiorenni e minorenni.

Sovrapposta: quando esiste almeno un'istanza dell'entità padre che appartiene a più categorie contemporaneamente. Un esempio in questo caso potrebbe essere `Persona`

`<- Studente, Lavoratore`, in quanto una persona può contemporaneamente essere uno studente ma anche un lavoratore.

Concluso il discorso sui costrutti che riguardano il modello E-R, spendiamo due parole anche per parlare della documentazione da associare quando si sviluppa un modello concettuale.

Dizionario delle entità: consiste in una tabella in cui si riportano le entità descritte nel modello assieme ad alcune informazioni importanti riguardanti queste: **nome, breve descrizione, attributi, identificatore univoco**.

Dizionario delle associazioni: come quello delle entità, si tratta di una tabella volta a descrivere tutte le associazioni presenti nel modello. Si includono informazioni come: **nome, breve descrizione, entità coinvolte, attributi dell'associazione**.

Vincoli non esprimibili: il modello concettuale è un ottimo strumento per rappresentare la realtà di riferimento del problema, tuttavia non è abbastanza esaustivo per quanto riguarda eventuali vincoli stabiliti sulla base di dati. Nell'ottica di una base di dati aziendale si potrebbe, ad esempio, impedire che un dirigente abbia uno stipendio inferiore a quello di un impiegato normale (ci rendiamo conto che è impossibile esprimere tale vincolo sul modello E-R).

5

Progettazione logica

Come seconda fase nella progettazione di una base di dati abbiamo la progettazione logica, in cui ci avviciniamo maggiormente all'architettura del DBMS e per questo bisogna definire il modello con cui si voglio opportunamente memorizzare i dati.
In questo corso faremo esclusivamente uso del modello relazionale, che a breve vedremo con maggior dettaglio.

5.1 Il modello relazionale

Il modello relazionale si basa sul concetto matematico di **relazione**: tali relazioni si traducono in maniera naturale in tabelle (infatti talvolta i due termini potrebbero essere scambiati). Prima di passare alla definizione formale di questo modello, vediamo alcune definizioni di oggetti matematici che ci serviranno:

Definizione 5.1.1: Dominio

Un **dominio** è un insieme potenzialmente infinito di valori. Ricordando lo studio di una funzione, il dominio di questa era l'insieme dei valori $x \in \mathbb{R}$ a cui la funzione associava un valore.

Possiamo scegliere anche esempi più comuni di dominio: l'insieme dei numeri naturali, l'insieme dei valori in un sistema di numerazione binario (banalmente solo 0 e 1), l'insieme delle stringhe lunghe 20 caratteri, ...

Definizione 5.1.2: Prodotto cartesiano

Siano $D_1, D_2, \ldots, D_k$ domini non necessariamente distinti. Il **prodotto cartesiano** tra tali domini è denotato come

$$D_1 \times D_2 \times \cdots \times D_k$$

che, insiemisticamente, può essere visto come quella sequenza ordinata di valori in cui ogni valore è preso da un dominio diverso.

$$\{(v_1, v_2, \ldots, v_k) | v_1 \in D_1, v_2 \in D_2, \ldots, v_k \in D_k\}$$

Definizione 5.1.3: Relazione matematica

Una **relazione**, in matematica, è un qualsiasi **sottoinsieme del prodotto cartesiano di uno o più domini**.

Il **grado di una relazione** è il numero di domini coinvolti nel prodotto cartesiano che hanno prodotto quella relazione (dunque se abbiamo il prodotto cartesiano tra 5 domini e prendiamo una relazione su quel prodotto, il suo grado sarà 5).

Gli elementi di una relazione sono detti **tuple** o **n-uple**, e il numero di n-uple di una relazione rappresenta la sua **cardinalità**.
L'ordine fra gli elementi di una tupla (che sono presi ognuno da ogni dominio, in modo **ordinato**) è **importante**, mentre è irrilevante l'ordine delle tuple di una relazione (essendo essa un insieme).

Supponiamo di avere due domini: $D_1 = \{\text{bianco, nero}\}, D_2 = \{1, 2, 3\}$.
Il prodotto cartesiano sarà

$$D_1 \times D_2 = \{\text{(bianco, 1), (bianco, 2), (bianco, 3), (nero, 1), (nero, 2), (nero, 3)}\}$$

Presa una relazione $R = \{\text{(bianco, 1), (nero, 1), (nero, 3)}\}$, questa avrà grado 2 (in quanto è stata generata dal prodotto cartesiano di 2 domini) e cardinalità 3 (in quanto ha 3 elementi).

Forniamo adesso dell'altra notazione riguardante relazioni e tuple. Sia R una relazione di grado k. Sia $t \in R$. Sia $i \in \{1, \ldots, k\}$. Denotiamo con $t[i]$ oppure $t.i$ l'**i-esima componente di t**. Per l'esempio di prima, supponiamo $t = \{(\text{nero}, 1)\}$. Abbiamo che t[1] = nero, t[2] = 1, t[1,2] = (nero, 1).

Abbiamo già detto che nei database relazionali c'è una fortissima corrispondenza tra il concetto di tabella e quello di relazione. Il problema è che se ci viene data una tabella così senza descrizioni, decifrarne il contenuto diventa quasi impossibile.

E' per questo che nasce l'importanza di definire degli attributi nelle tabelle.

Definizione 5.1.4: Attributo

Un **attributo** è definito da un **nome** `A` e dal **dominio** di tutti i valori che `A` può assumere.

Definizione 5.1.5: Schema di relazione e tuple

- Sia `R` un insieme di attributi diversi (**schema di relazione**, sarebbe l'intestazione della tabella). Una n-upla su `R` è una **funzione che associa ad ogni attributo A in R un elemento del dominio di A**.
  In poche parole, una n-upla, quella che informalmente chiameremmo *riga della tabella* o *istanza della tabella* viene matematicamente definita come una funzione che ad ogni colonna associa il suo effettivo valore, preso appunto dal dominio di valori consentiti dall'attributo.
- Se `t` è una n-upla su `R` ed `A` è un attributo di `R`, indichiamo con `t(A)` il valore assunto dalla funzione `t` in corrispondenza dell'attributo `A`.
  Sempre in parole molto povere, facendo `t(A)` ci viene restituito il valore che l'attributo `A` assume nella riga `t`.
- Se $Y \subseteq X$, dove X è lo schema di una relazione R, denotiamo con t[Y] l'insieme dei valori assunti dagli attributi $y \in Y$ nella tupla t.

Infine, possiamo definire lo **schema di una base di dati relazionale** come l'insieme di tutti gli schemi di relazione che hanno nomi differenti.

Città	Regione	Popolazione
Roma	Lazio	3000000
Milano	Lombardia	1500000
Genova	Liguria	800000
Pisa	Toscana	150000

Tabella 5.2: Dati relativi a città, regione di appartenenza e popolazione

5.1.1 Chiavi di una relazione

Abbiamo già implicitamente parlato di chiavi quando, nel modello concettuale, citavamo gli identificatori di un'entità. Avendo però ora definito più concetti, possiamo fornire una definizione leggermente più formale di chiave.

Definizione 5.1.6: Chiave

Una **chiave** è un attributo o un insieme di attributi che identifica univocamente una tupla.

Supponendo che `X` sia un insieme di attributi che formano una chiave, valgono le seguenti proprietà:

- Se $t_1, t_2, t_1 \neq t_2$ tuple di una relazione R, allora $t_1[X] \neq t_2[X]$.
- X è il più piccolo insieme che soddisfa la proprietà precedente.

La chiave composta dal minor numero di attributi possibili ad identificare una tupla si dice **chiave primaria**. Questa gode di due fondamentali proprietà:

- Non può avere valori nulli.
- Consente di mettere in relazione tabelle diverse.

5.2 Ristrutturazione del modello concettuale

Come mai la fase di ristrutturazione del modello concettuale appartiene alla progettazione logica?
Ricordando quando detto nei primi capitoli, è molto importante progettare una base di dati mantenendo i paradigmi di indipendenza fisica e logica. Se nella prima fase di progettazione concettuale abbiamo descritto la realtà in modo completamente astratto e indipendente dalla memorizzazione fisica dei dati, con questa fase di ristrutturazione dovremo andare ad analizzare il modello concettuale già nell'ottica di voler organizzare i dati in tabelle e relazioni tra queste. Questo avviene in quanto l'obiettivo principale della progettazione logica è quello di **tradurre il modello concettuale in uno schema logico che rappresenti i dati nella stessa maniera**. Questa fase di ristrutturazione si articola in 4 attività principali.

5.2.1 Analisi delle ridondanze

Nello schema E-R, una ridondanza è la ripetizione di un'informazione utile ma derivabile da altre, ad esempio:

- Tramite operazioni di interrogazione sulla base di dati.
- Da altri attributi dell'entità stessa. Ad esempio, non ha senso inserire gli attributi `IVA, PrezzoNetto, PrezzoLordo` in un'entità `Prodotto`, in quanto questi attributi sono calcolabili a partire da almeno due di loro.

- Ridondanza derivata da più associazioni sulle stesse entità, ma **solo se le istanze delle associazioni rappresentano lo stesso risultato**.

5.2.2 Eliminazione delle generalizzazioni

Il modello relazionale abbiamo visto essere in grado di rappresentare solo tabelle, non costrutti di ereditarietà come classi e oggetti. Per questo motivo è necessario eliminare tutte le gerarchie rappresentate nel modello concettuale, e ciò può essere fatto in 3 modi differenti:

1. **Accorpamento delle figlie nel padre (Roll-up)**

   - **Cosa si fa:** Si elimina ogni entità figlia. L'entità padre assorbe tutti gli attributi specifici delle entità figlie e tutte le relazioni a cui partecipavano. Viene inoltre introdotto un apposito attributo *selettore* (discriminatore di tipo), ad esempio `TipoPersona = {'Studente', 'Docente'}`, per identificare a quale sottocategoria appartiene ciascuna istanza.
   - **Vantaggi:**
     - Estrema semplicità nella gestione delle proprietà e operazioni comuni.
   - **Svantaggi:**
     - Introduzione di valori nulli (`NULL`): se un'istanza è uno studente, tutti gli attributi propri del docente assumeranno valore nullo.
     - Potenziale spreco di spazio su memoria secondaria e degradazione dell'integrità dei vincoli a livello di colonna.
   - **Condizioni e casi d'uso:** Sempre applicabile senza vincoli di tipo di gerarchia. È la scelta ideale quando le entità figlie possiedono **pochi attributi specifici** e **poche relazioni proprie**, e la maggior parte delle operazioni coinvolge l'insieme complessivo delle istanze (padre e figlie insieme).

2. **Accorpamento del padre nelle figlie (Roll-down)**

   - **Cosa si fa:** Si elimina l'entità padre e si mantengono unicamente le entità figlie. Ciascuna entità figlia eredita tutti gli attributi della classe padre (inclusa la chiave primaria) e tutte le relazioni a cui il padre prendeva parte.
   - **Vantaggi:**
     - Assenza totale di valori nulli dovuti alla gerarchia: ogni relazione contiene unicamente attributi significativi per le sue istanze.
     - Accessi veloci e selettivi per tutte le operazioni mirate a una singola categoria figlia.
   - **Svantaggi:**

  - Ridondanza concettuale: gli attributi e le relazioni originarie del padre vengono replicati nelle specifiche di ogni singola figlia.
- **Condizioni e casi d'uso:** Questo metodo è applicabile in modo naturale **solo se la generalizzazione è TOTALE**. Se fosse parziale, esisterebbero istanze del padre non appartenenti ad alcuna figlia, le quali andrebbero irrimediabilmente perdute eliminando l'entità radice. È inoltre altamente raccomandato che sia **esclusiva** (disgiunta), per evitare di duplicare la stessa entità in più tabelle figlie. Si predilige quando le entità figlie hanno molti attributi distinti e le interrogazioni avvengono quasi sempre separatamente su ciascuna figlia.

3. **Sostituzione della generalizzazione con associazioni**

- **Cosa si fa:** Si mantengono separate sia l'entità padre sia le entità figlie. La gerarchia viene rimpiazzata da associazioni binarie uno a uno (1:1) tra il padre e ciascuna entità figlia. Ogni tabella figlia adotta come chiave primaria la stessa chiave del padre, fungendo contemporaneamente da chiave esterna verso quest'ultimo.
- **Vantaggi:**
  - Massima flessibilità e generalità: è sempre applicabile, indipendentemente dal fatto che la gerarchia sia totale/parziale o esclusiva/sovrapposta.
  - Rispetto rigoroso della semantica originale, assenza di campi nulli anomali e nessuna duplicazione di tuple.
- **Svantaggi:**
  - Impatto sulle prestazioni: per ricostruire l'insieme completo delle informazioni di un'istanza figlia è sempre necessaria una complessa operazione di congiunzione tra le informazioni del padre e quelle della singola classe.
- **Condizioni e casi d'uso:** Sempre applicabile. È la soluzione di riferimento in caso di generalizzazioni **parziali** e/o **sovrapposte**, oppure quando sia l'entità genitore che quelle figlie dispongono di numerosi attributi e relazioni distinti e sussistono accessi eterogenei (sia generalisti che mirati alle sottoclassi).

5.2.3 Partizionamento/accorpamento di entità e associazioni

Si tratta forse dell'operazione più ambigua, in quanto non dipende tanto dall'astrazione che abbiamo fatto del problema, bensì di come devono essere acceduti i dati dall'applicazione. Questa fase può svolgersi secondo 4 diverse modalità a seconda del caso.

1. **Partizionamento verticale di entità**: supponiamo di avere un'entità `Impiegato`, che contiene sia i dati anagrafici che lavorativi. Supponiamo anche che la nostra applicazione debba accedere separatamente ai dati lavorativi e quelli anagrafici. In questo caso potremmo pensare di suddividere l'entità `Impiegato(codice, cognome, indirizzo, data_nascita, livello, stipendio, ritenute)` in due entità separate `DatiAnagrafici(codice,`

`cognome, indirizzo, data_nascita), DatiLavorativi(stipendio, livello, ritenute)`. In seguito queste due entità verranno collegate da un'apposita associazione che le mette in relazione in quanto fanno riferimento allo stesso impiegato.

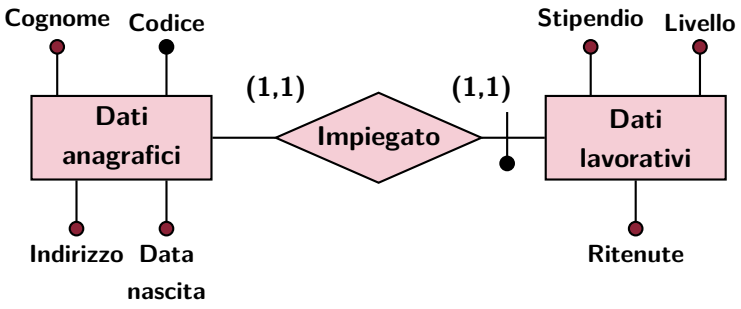
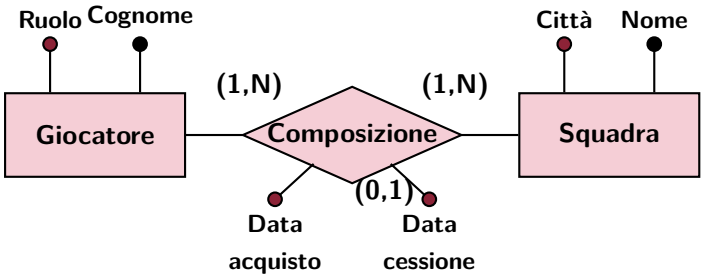


Figura 5.1: Esempio di partizionamento verticale

2. **Partizionamento orizzontale di associazioni**: supponiamo di avere una base di dati che mantiene le informazioni riguardo ai giocatori di una squadra da calcio, sia presenti che passati.



In un caso come questo, supponiamo ci sia richiesto di far accedere separatamente ai dati dei giocatori passati e di quelli correnti. Potremmo dividere dividere l'associazione `Composizione` in due associazioni tra `Composizione attuale` e `Composizione passata`, in modo da gestire separatamente i due casi.

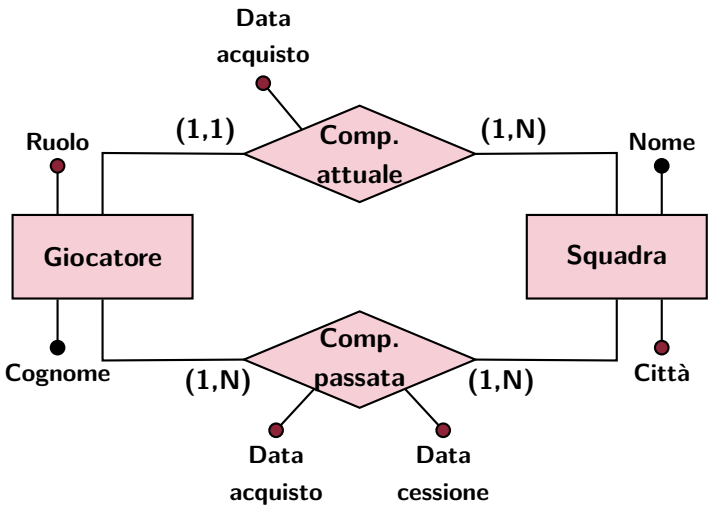
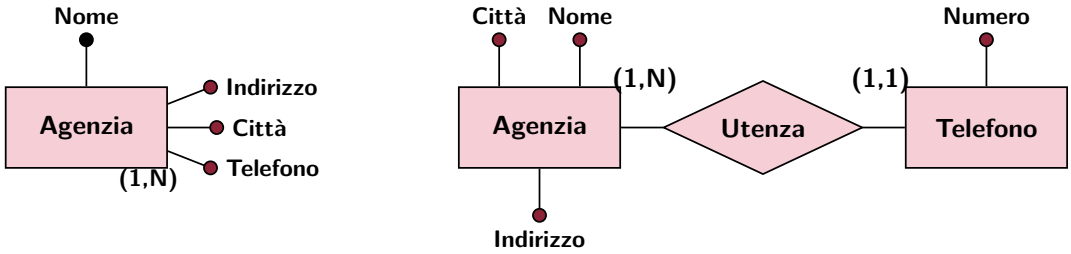


Figura 5.2: Partizionamento orizzontale di associazioni.

3. **Eliminazione di attributi multivalore**. Possiamo avere due istanze di attributi multivalore:

    - Un attributo che ha cardinalità (0, N) oppure (1, N). In tal caso, l'attributo deve diventare un'entità a sé stante, associata all'entità di cui era attributo.



    - In caso di un attributo che comprende più informazioni (come il precedente esempio di `Indirizzo = Via, Numero civico, CAP`), questo deve essere separato nei suoi attributi atomici.

5.2.4 Scelta degli identificatori primari

Come abbiamo visto prima, la **chiave primaria** di un'entità è il più piccolo insieme di attributi che identifica in modo univoco ogni tupla. Guardando con occhio di riguardo anche la progettazione logica, possiamo aggiungere un paio di vincoli:

- Preferibilmente scegliere tra identificatori interno. Se non possibile, è consentito aggiungere un codice artificiale che identifichi la tupla.
- Deve essere un attributo che viene utilizzato nelle operazioni più frequenti ed importanti.

5.3 La traduzione da schema concettuale a logico

Dopo aver rivisto e ristrutturato il modello concettuale con un occhio di riguardo per quella che sarà la rappresentazione dei dati secondo il modello logico, è arrivato il momento di capire come tradurre effettivamente i costrutti del modello concettuale in quelli del modello logico scelto (quello relazionale, nel nostro caso).

5.3.1 Le entità

Di una cosa siamo certi: le entità diventeranno le **tabelle** (o relazioni) del nostro modello logico. Ogni tabella dovrà anche preservare l'insieme degli attributi descritti dal modello concettuale.

5.3.2 Le associazioni

Per le associazioni, ovvero le effettive relazioni che vengono stabilite tra due o più schemi, il discorso è leggermente più delicato, in quanto non esiste un modo strettamente deterministico per rappresentarle.
Prima di discutere separatamente ogni tipo di associazione che possiamo incontrare all'interno dello schema E-R e la rispettiva traduzione nel modello logico, dobbiamo introdurre un fondamentale concetto delle basi di dati relazionali.

Definizione 5.3.1: Vincolo di integrità referenziale

Si tratta di uno dei tanti vincoli di integrità che si stabiliscono all'interno di una base di dati relazionale.
In particolare, questo vincolo stabilisce un **collegamento logico tra due schemi attraverso uno o più attributi con dominio comune**.

Parlando in modo più formale, supponiamo di avere una tabella R_1 con un insieme di attributi X. Prendiamo adesso un'altra tabella R_2. Stabilendo un vincolo di integrità referenziale fra le tabelle R_1 ed R_2 su X significa **imporre** che i valori assunti da X in R_1 debbano comparire come valori della chiave primaria di R_2. Tale insieme di attributi, nella tabella di "arrivo" R_1 viene detto **chiave esterna** o **foreign key**.

In parole povere, stiamo dicendo che se in R_1 abbiamo un insieme di attributi X, affinché il vincolo di integrità referenziale con R_2 sia valido dobbiamo avere la certezza che R_2 contenga i valori di chiave primaria espressi da X in almeno una n-upla.

Avendo definito il concetto di chiave esterna, possiamo procedere ad analizzare uno ad uno i tipi di associazione.

5.3.2.1 Associazioni: Molti a molti

Un'associazione *molti a molti* si traduce come una vera e propria **tabella** che ha come chiave **l'insieme degli attributi chiave delle tabelle coinvolte nell'associazione**. Inoltre, tale tabella include anche eventuali attributi dell'associazione.

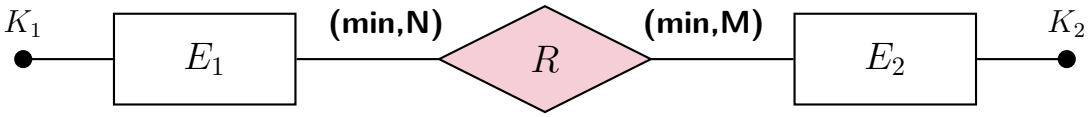


Figura 5.3: Modello concettuale generico di una associazione molti a molti.

Secondo quanto detto prima, il frammento di schema mostrato si traduce con le seguenti relazioni:

E1(**K1**, attributi di E1...)

E2(**K2**, attributi di E2...)
R(**K1**, **K2**, attributi dell'associazione...)

E stabilendo anche i vincoli di integrità referenziale:

- Tra K1 in R e K1 in E1.
- Tra K2 in R e K2 in E2.

Generalmente, è consigliato rinominare gli attributi che costituiscono un vincolo di integrità referenziale, dando come nome un qualcosa di indicativo della tabella di provenienza (es: se l'attributo di partenza si chiama `Codice` e proviene dalla tabella `Impiegato`, potremmo chiamare la chiave esterna come `FK_Codice_Impiegato` oppure semplicemente `Impiegato`. In caso di chiavi composte da più attributi, possiamo rinominarle come `Impiegato1`, `Impiegato2`, ...).

Particolare attenzione deve essere posta sulle associazioni molti a molti ricorsive, in quanto in quel caso è di vitale importanza il nome delle chiavi esterne.

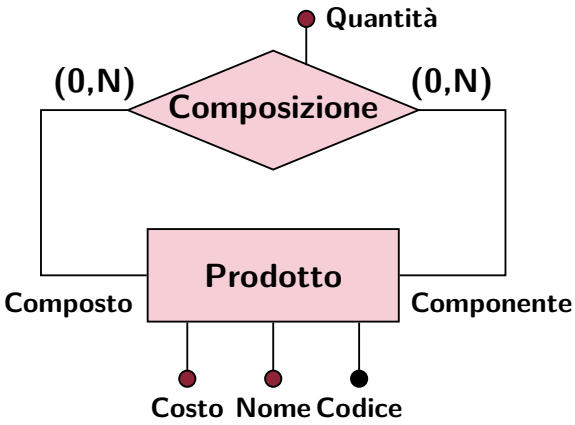


Figura 5.4: Esempio di associazione molti a molti ricorsiva.

La derivazione di questo modello concettuale sarà

Prodotto(**Codice**, Nome, Costo)
Composizione(**CodiceComposto**, **CodiceComponente**, Quantita)

In questo caso i due codici presenti nella tabella `Composizione` non saranno altro che codici di prodotti però inseriti come chiavi esterne.

Un ultimo esempio sulle associazioni molti-a-molti per associazioni n-arie:

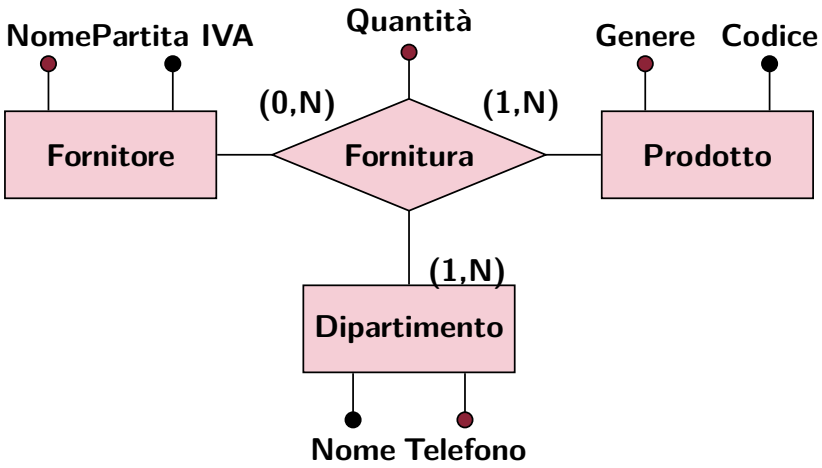


Figura 5.5: Esempio di associazione n-aria molti-a-molti.

In questo caso si deriva come ci si aspetterebbe, ma si è scelto di mostrare un esempio per evitare che il lettore si cogliesse impreparato. Il fulcro è sempre creare una tabella per l'associazione e mettere al suo interno aventuali attributi dell'associazione e, come chiave primaria, l'insieme

delle chiavi di ogni tabella coinvolta.
La derivazione risulta dunque:

Fornitore(**PartitaIVA**, Nome)
Prodotto(**Codice**, Genere)
Dipartimento(**Nome**, Telefono)
Fornitura(**Fornitore**, **Prodotto**, **Dipartimento**, Quantita)

5.3.2.2 Associazioni: Uno a molti

Per questo tipo di associazioni, si può procedere in due modalità diverse:

1. **Creare una tabella per l'associazione**, al cui interno si metterà:
   - Come **chiave primaria**, la **chiave primaria della tabella che partecipa all'associazione con cardinalità (min,1)** (che sarà anche chiave esterna, legata da vincolo di integrità referenziale).
   - Come **chiave esterna**, la **chiave primaria della tabella che partecipa all'associazione con cardinalità (min,N)**.
2. **Inserire nella tabella che partecipa con cardinalità (min,1) un vincolo di integrità referenziale alla chiave primaria della tabella che partecipa con (min, N)**. Tutti gli eventuali attributi dell'associazione andranno a finire nella tabella con cardinalità (min,1).

Vediamo di seguito un esempio di modello concettuale, e discutiamo le due possibili traduzioni nel modello logico.

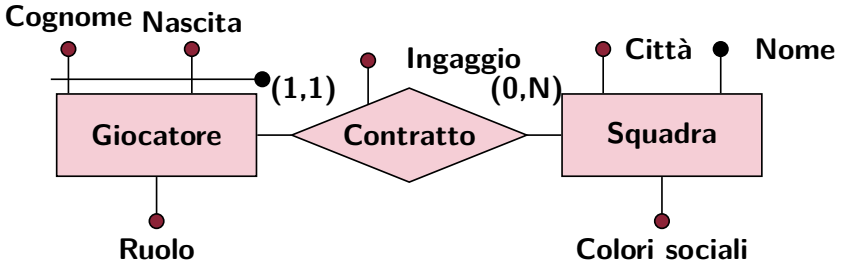


Figura 5.6: Esempio di associazione uno-a-molti tra due entità

Seguendo il primo modo descritto, ovvero quello di creare una nuova tabella, la derivazione sarà la seguente:

Giocatore(**Cognome**, **Nascita**, Ruolo)
Squadra(**Nome**, ColoriSociali, Citta)
Contratto(**Cognome**, **Nascita**, *Squadra*, Ingaggio)

Il **vantaggio** di questa rappresentazione è che, creando una tabella apposita per la relazione tra le due tabelle, *non dovremmo preoccuparci di eventuali valori nulli o di istanze delle tabelle che non hanno corrispondenza nell'associazione*. Il principale svantaggio consiste invece nel fatto che stiamo definendo un'ulteriore schema per la nostra relazione, dunque un ulteriore tabella da indicizzare per il DBMS (parleremo dell'efficienza più avanti nel corso).

Se invece volessimo derivare il precedente modello nel secondo modo, otterremmo:

Squadra(**Nome**, Citta, ColoriSociali)
Giocatore(**Nome**, **Nascita**, Ruolo, *Squadra*, Ingaggio)

In questa rappresentazione, all'interno della tabella `Giocatore` (quella che partecipa con cardinalità (1,1) all'associazione) mettiamo un vincolo di chiave esterna alla chiave primaria della tabella `Squadra` e l'attributo dell'associazione `Ingaggio`.

Questo metodo rappresenta la soluzione più semplice e compatta, in quanto non richiede la creazione di tabelle aggiuntive ma si limita ad aggiungere attributi ad una già esistente. Lo svantaggio è legato alla possibile presenza di valori nulli all'interno dell'entità *debole*: se questa partecipasse all'associazione con una cardinalità (0,1) significherebbe che ci potrebbero essere istanze della tabella che non hanno correlazione con quella associata. Per tabelle con un gran numero di record, si potrebbe verificare un ingente spreco di memoria.

Definizione 5.3.2: Entità debole

Un'entità (o successivamente una tabella nel modello relazionale) si definisce **debole** quando **la sua esistenza dipende dall'esistenza di una o più entità**.
In genere, sono considerate deboli quelle entità al cui interno sono presenti chiavi esterne di un'altra entità, in quanto per essere definite devono prima essere definite le entità da cui provengono tali chiavi esterne.

Andiamo a vedere anche per le associazioni uno-a-molti le casistiche di ricorsività oppure n entità coinvolte.

Nel caso di questa associazione, si vogliono spendere due parole su entrambe le traduzioni in quanto potrebbero presentare delle criticità:

1. **Creazione di una tabella per la relazione**, e dunque si avrà lo schema

   Persona(**CF**, Nome, Cogn)
   Paternita(**Figlio**, *Padre*)

   Si presti particolare attenzione alla scelta della chiave primaria nella tabella `Paternita`:

   - Se avessimo scelto solo **Padre** ci saremmo trovati nella situazione in cui un padre può avere al più un figlio, chiaramente errata per lo schema rappresentato.

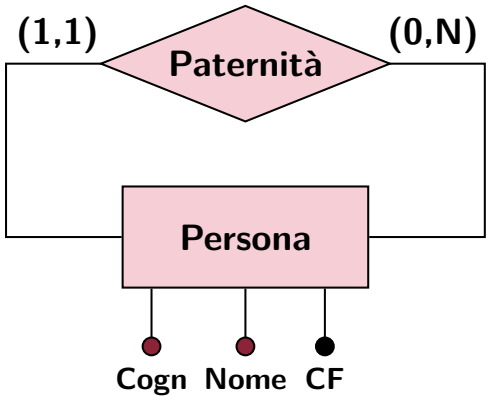


Figura 5.7: Associazione uno-a-molti ricorsiva.

- Se avessimo scelto come chiave la coppia **Figlio, Padre** ci saremmo potuti trovare nella situazione in cui un figlio è associato a più di un padre (in quanto, fissando il figlio e modificando il padre si possono generare tuple differenti).

2. **Unificazione in una sola tabella**. In questo caso lo schema sarebbe stato semplicemente Persona(**CF**, Nome, Cogn, *Padre*), dove *Padre* rappresenta una chiave esterna.

   Si noti una criticità di questa implementazione: per come definito prima, il vincolo di integrità referenziale impone che i valori associati alla chiave esterna siano valori **esistenti** nella tabella di partenza. Siccome in questo caso l'associazione è ricorsiva, la tabella di partenza è la stessa di quella di arrivo. Dunque questo ci mette nella condizione in cui ci è sostanzialmente **impossibile** (almeno all'inizio) inserire delle persone all'interno della tabella, in quanto non potremmo associarle un padre.
   Ci sono due possibili soluzioni:

   (a) Disabilitare temporaneamente i controlli di integrità del DBMS, per permettere di inserire delle persone e successivamente riabilitarli per creare la corrispondenza effettiva.

   (b) Impostare il valore della chiave esterna come non obbligatorio (**RISCHIOSO!!!**) e inserire il valore della chiave esterna solo *dopo aver effettuato sufficienti inserimenti*.

Vediamo di seguito un esempio anche con le associazioni n-arie molti-a-molti.

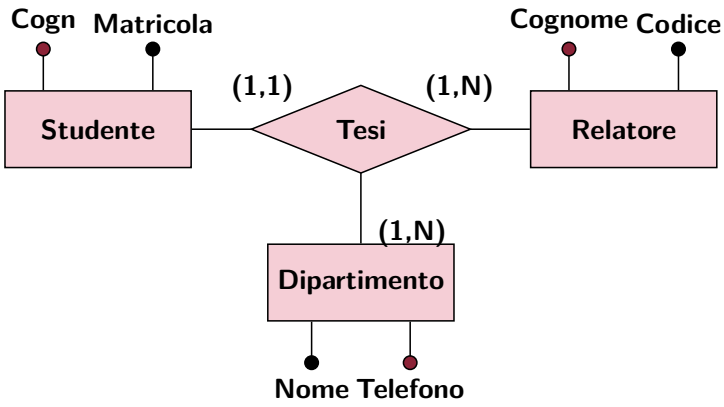


Figura 5.8: Associazione ternaria molti-a-molti.

Il trucco per le associazioni n-arie è *leggerle a due a due*, dunque prendendo una coppia di entità e leggere l'associazione tra queste. Fare poi questo lavoro per tutte le entità coinvolte. Anche in questo caso la traduzione può avvenire in due modi:

1. **Creando la tabella per l'associazione**, ottenendo il seguente schema della base di dati:

   Studente(**Matricola**, Cogn)
   Relatore(**Codice**, Cogn)
   Dipartimento(**Nome**, Telefono)
   Tesi(**Studente**, *Relatore*, *Dipartimento*)

   Ci accorgiamo però che le tabelle `Studente` e `Tesi` hanno la stessa chiave primaria, e dunque conviene unificarle.

2. **Unificare tutto in un'unica tabella**, ottenendo lo schema:

   Relatore(**Codice**, Cogn)
   Dipartimento(**Nome**, Telefono)
   Studente(**Matricola**, Cogn, *Relatore*, *Dipartimento*)

5.3.3 I valori NULL

I valori NULLI rappresentano mancanza di informazione oppure che un'informazione non è applicabile a quella n-upla. Vediamo un esempio con un'informazione che usiamo tutti i giorni: il numero di telefono.
In una base di dati, il numero di telefono di una persona potrebbe assumere un valore nullo in diverse circostanze:

- La persona effettivamente non possiede un telefono cellulare e dunque neanche un numero.

- Non sappiamo se la persona abbia un numero di telefono.
- La persona ha un numero di telefono ma non lo conosciamo.

Nell'ottica della progettazione logica, dobbiamo stabilire questo vincolo (indicando che l'attributo può assumere valori NULL), ma in generale si tende a non utilizzare questa pratica, in quanto per la definizione di tupla questa deve possedere un valore per ogni attributo nello schema. Infatti, una possibile tecnica potrebbe essere quella di attribuire un valore di **default**.
Ci rendiamo anche conto di come molti campi debbano categoricamente essere **obbligatori**. Si tratta solitamente dei campi che contengono informazioni fondamentali per l'identificazione della tupla (come ad esempio la matricola di uno studente, che ci permette di identificarlo univocamente) oppure il voto di uno studente ad un appello di esame (non usiamo questa informazione per identificare lo studente, ma si tratta di un'informazione fondamentale in una base di dati universitaria).

Entrando più nello specifico del valore NULL, possiamo dire che:

- Si tratta di un valore **polimorfo**, cioè che non appartiene nello specifico a nessun dominio, ma può sostituire valori in qualsiasi dominio (ad esempio, se abbiamo degli attributi di tipo intero e stringhe, possiamo comunque inserire NULL come valore in quanto sarà compatibile con entrambi).
- Due valori NULL, anche se sullo stesso dominio, sono considerati **diversi**.
- **IMPORTANTE**! NULL $\neq 0$.

5.4 Vincoli di integrità

Definizione 5.4.1: Vincolo di integrità

Un **vincolo di integrità** è una proprietà che deve essere categoricamente rispettata da **ogni istanza** della base di dati, in quanto è strettamente legata allo schema di una relazione.

Un'istanza della base di dati si dice **corretta** se soddisfa tutti i vincoli di integrità associati al suo schema.

I vincoli di integrità si possono suddividere in due tipologie:

Vincoli di integrità intra-relazionali: sono **vincoli che si stabiliscono sugli attributi o sulle tuple di una singola relazione**. Alcuni esempi di questi sono:

- Vincolo di chiave primaria: attributo obbligatorio e mai nullo. Identifica univocamente una tupla all'interno della relazione.

- Vincolo di dominio: quando si restringe il dominio di un particolare attributo.
- Vincolo di unicità: stabilisce che il valore di un attributo in una tupla t deve essere univoco e non preso da nessun'altra tupla.
- Vincolo di obbligatorietà: stabilisce che un attributo deve necessariamente avere un valore.

Vincoli di integrità inter-relazionali: sono **vincoli che si stabiliscono tra relazioni diverse**, "collegando" ad esempio attributi con lo stesso dominio e creando un legame in base al valore assunto dai due attributi. Un esempio è il vincolo di chiave esterna che abbiamo citato qualche sezione fa.

Facciamo alcuni esempi concreti di violazione di vincoli, oppure di casi in cui si potrebbero imporre dei vincoli.

CodiceImp	Cognome	Ruolo	Assunzione	Dip
COD1	Rossi	Analista	1795	01
COD2	Bianchi	Analista	1990	05
COD2	Neri	Amministratore	1985	01

CodiceDip	Nome
01	Progettazione
02	Amministrazione

Considerando quest'esempio di base di dati, ci sono molti problemi: vediamo dove risiedono.

1. L'impiegato `COD1` risulta avere anno di assunzione 1795. Ciò è chiaramente impossibile, e una possibile soluzione potrebbe essere stabilire un anno di assunzione minimo (*vincolo di dominio*).

2. L'impiegato "Bianchi" risulta appartenere al dipartimento di codice "05", che è tuttavia inesistente, in quanto non presente in alcuna tupla della tabella `Dipartimenti`. Questo si potrebbe risolvere stabilendo un *vincolo di chiave esterna* tra l'attributo `Dip` di `Impiegato` e l'attributo `CodiceDip` di `Dipartimenti`.

3. Vi è una duplicazione del codice identificativo per gli impiegati "Bianchi" e "Neri". Ciò si potrebbe risolvere imponendo un *vincolo di unicità* sui valori della colonna `CodiceImp`.

Ovviamente esistono tanti altri tipi di vincoli, e altrettanti modi per definirli nello schema della base di dati, ma li vedremo più nel dettaglio durante la creazione effettiva delle relazioni.

6

Il linguaggio SQL

Al fine di creare i database, le tabelle, stabilire i vincoli tra queste ed effettuare operazioni di interrogazione sui dati era necessario un apposito linguaggio con cui fare tutto questo.
Nasce così **SQL (Structured Query Language)**, il linguaggio diventato ormai di riferimento per tutti i DBMS moderni.

Definizione 6.0.1: Sottolinguaggi di SQL

Il linguaggio SQL è un grande contenitore di moltissime istruzioni di vario genere. Nonostante la suddivisione logica di queste istruzioni non sia importante ai fini del calcolo, si tratta comunque di una nozione utile ai fini della progettazione o dell'uso di un DBMS.
Le varie istruzioni del linguaggio SQL si suddividono in 4 sottolinguaggi:

- **DDL (Data Definition Language)**: contiene tutte le istruzioni che permettono la creazione/modifica/eliminazione di database e tabelle (es: `CREATE TABLE (...)`, `CREATE DATABASE ..., ...`).
- **DML (Data Manipulation Language)**: comprende le istruzioni utili alla manipolazione dei dati all'interno delle tabelle, permettendo ad esempio istruzioni di inserimento, modifica o cancellazione di un dato (es: `INSERT INTO ... VALUES (...)`).
- **DQL (Data Query Language)**: si tratta del linguaggio che comprende tutte le istruzioni utili all'interrogazione dei dati (es: `SELECT ... FROM ... WHERE ...`).
- **DCL (Data Control Language)**: gestisce i permessi utente e limita l'accesso ai dati (`GRANT, REVOKE, ...`).

6.1 Domini di dati

Come abbiamo già discusso nella formalizzazione del modello relazionale per la memorizzazione dei dati, ad ogni attributo è associato un dominio di valori che possono essere assunti da esso. Nel linguaggio SQL ci vengono forniti dei domini di dati predefiniti, ma ci è anche dato modo di crearne di nostri.
Di seguito una tabella in cui riassumiamo i domini principali forniti dal linguaggio SQL. Chiaramente ne esistono molto di più, ma questi sono gli essenziali nella progettazione di una base di dati standard.

Nome in SQL	Descrizione
`CHAR(size)`	Rappresenza una stringa di lunghezza fissa di `size` caratteri. Il massimo di caratteri rappresentabili è 255.
`VARCHAR(size)`	Rappresenta una stringa a lunghezza variabile il cui numero massimo di caratteri è 65535.
`INT`	Rappresentazione standard di un valore intero. Ci sono anche altri tipi di dato per cui varia il range di rappresentazione dei valori.
`FLOAT`	Memorizza valori decimali con una "bassa precisione".
`DOUBLE`	Il tipo di dato più preciso per memorizzare valori decimali.
`BLOB/CLOB`	Predisposti a contenere grandi immagini e testi, ma non possono essere oggetto di interrogazioni.
`BOOLEAN/BOOL`	Modo standard per memorizzare valori come TRUE (tutti i numeri diversi da 0) o FALSE (unicamente 0).
`DATE`	Memorizza una data nel formato "YYYY-MM-DD".
`TIME`	Memorizza l'ora nel formato "hh:mm:ss".

`DATETIME`	Una combinazione di `DATE` e `TIME` che si presenta nel formato "YYYY-MM-DD hh:mm:ss".

Per creare invece dei domini personalizzati, si usa l'istruzione `CREATE DOMAIN`. Per quanto la sintassi ora come ora possa non essere falmiliare, viene lasciato di seguito un esempio:

```
CREATE DOMAIN AnnoAssunzione AS INT DEFAULT NULL CHECK (value >= 1990)
```

Provando a spiegare la precedente istruzioni, stiamo

1. Usando le istruzioni `CREATE DOMAIN` per "annunciare" di voler creare un dominio personalizzato.
2. Definendo il nome del dominio (AnnoAssunzione).
3. Keyword `AS` per indicare che tutto ciò che segue saranno le istruzioni per la definizione del dominio (tutto ciò che ci risparmieremo di scrivere ogni volta).
4. Definiamo il tipo di dominio a cui applicare le restrizioni. In questo caso `INT`.
5. Stabiliamo un valore di default, cioè in caso il dato non sia ancora stato inserito nella tabella. In questo caso abbiamo impostato `DEFAULT NULL`.
6. Definiamo l'effettiva *restrizione* da applicare al tipo di dato di partenza. La parola chiave `CHECK` serve per indicare che il DBMS deve **controllare** che la seguente condizione racchiusa tra () sia rispettata.
7. `value` è un attributo specifico di ogni dominio di dato e rappresenta l'effettivo valore che si è provato ad inserire nella tabella. In questo caso vogliamo imporre che tutti i dati che apparterranno al dominio "AnnoAssunzione" debbano avere un valore >= 1990.

6.2 Definizione delle tabelle

Di seguito vediamo come definire una tabella in SQL (mediante istruzioni del linguaggio DDL) e analizzeremo tutti i *constraint* che possiamo imporre sullo schema di questa.
Innanzitutto, partiamo dalle basi: la creazione di una tabella avviene con l'istruzione `CREATE TABLE nomeTabella (...);` Mostriamo di seguito un esempio di creazione di due tabelle e della creazione dei vincoli inter-relazionali e intra-relazionali.

```
1    CREATE TABLE Dipartimento(
2      Codice CHAR(15) PRIMARY KEY,
```

```
    Nome CHAR(30) UNIQUE NOT NULL
);

CREATE TABLE Impiegato(
    Matricola CHAR(6) PRIMARY KEY,
    CF CHAR(16) UNIQUE NOT NULL,
    Cognome CHAR(30) NOT NULL,
    Stipendio DOUBLE DEFAULT 0,
    FK_Dipartimento CHAR(15) REFERENCES Dipartimento(Codice)
);
```

Analizziamo adesso ogni istruzione, spieghiamola e forniamo eventuali varianti di sintassi.

- In generale, la definizione di un attributo segue la sintassi `NOME_ATTRIBUTO DOMINIO CONSTRAINT`.
- Se vogliamo indicare che un attributo è chiave primaria di una relazione (ossia l'attributo che identifica univocamente ogni tupla della relazione) possiamo procedere in due modi:
  1. Come nell'esempio, scrivendo `PRIMARY KEY` come constraint dell'attributo.
  2. Prima della fine della definizione della tabella, possiamo scrivere `PRIMARY KEY (Attributo1, Attributo2, ...)` per definire una chiave primaria composta da uno o più attributi. Questa modalità è **obbligatoria** quando abbiamo relazioni in cui la chiave primaria è composta da più di un attributo, in quanto scrivere `PRIMARY KEY` come constraint su due attributi diversi risulterebbe in un errore.
- Per definire un vincolo di chiave esterna tra due relazioni possiamo anche qui procedere in due modi:
  1. Come nell'esempio, scrivendo `NOME_ATTRIBUTO TIPO REFERENCES TABELLA_DI_PROVENIENZA(NOME_ATTRIBUTO_DI_PROVENIENZA)`
  2. Utilizzando una sintassi simile a quella per le chiavi primarie composte, ovvero `FOREIGN KEY(NOME_ATTRIBUTO) REFERENCES TABELLA_DI_PROVENIENZA(NOME_ATTRIBUTO_DI_PROVENIENZA)`. Di nuovo, questa modalità è **obbligatoria** quando la chiave esterna ereditata dalla relazione sia una chiave primaria composta nella relazione di partenza.

Osservazione 6.2.1: Il constraint PRIMARY KEY

Il constraint `PRIMARY KEY` include implicitamente anche i vincoli `UNIQUE` (cioè che ogni valore memorizzato per quell'attributo deve essere univoco e non devono apparire valori duplicati) e `NOT NULL` (ovvero che non sono ammessi valori nulli per quel campo), oltre

all'indicizzazione da parte del DBMS (ma qui si scende al modello fisico).

Osservazione 6.2.2: Importanza dei domini nelle `FOREIGN KEY`

Come visto nell'esempio, affinché la creazione di un vincolo di integrità inter-relazionale tra due tabelle vada a buon fine è importante che l'attributo/gli attributi che si vogliono mettere in relazione abbiano **lo stesso dominio**. E ciò non vuol dire semplicemente che siano entrambi due `CHAR`, ma vuol dire ad esempio che se è stata configurata una dimensione per la stringa dei caratteri, questa debba essere rispettata (es: se nella tabella di partenza avevo `CHAR(16)`, anche in quella di arrivo deve essere `CHAR(16)`).